DESIGN QUESTIONS

# Design a TypeAhead / Search Autocomplete System

Trie, Top-K, Redis & Offline Pipeline

~ 30 minute read

© Study Guide — For personal use only

## What's Inside

**01** · Problem Statement & Clarifying Questions
**02** · Functional & Non-Functional Requirements
**03** · Capacity Estimation
**04** · High-Level System Architecture
**05** · Architecture Component Breakdown
**06** · Core Data Structure: Trie
**07** · Ranking & Scoring Formula
**08** · Trie Update Pipeline
**09** · Request Flow Deep Dive
**10** · Database Schema & Persistence
**11** · Partitioning Strategy
**12** · Failure Modes & Recovery
**13** · Interview Playbook (45 min)
**14** · Key Takeaways


© Study Guide — For personal use only

# 01 Problem Statement & Clarifying Questions

Scope, assumptions, and trade-offs

You are tasked with designing a search autocomplete system that powers type-ahead suggestions on a global scale. Users expect real-time, accurate, and personalized suggestions as they type. The system must be low-latency, highly available, and resilient to failures.

### Key Clarifications to Ask

| Question | Assumption |
|---|---|
| Scale? | 5 billion queries per day across all regions |
| Queries/user? | ~10 queries per day on average (active users) |
| Query latency SLA? | < 50ms p99, ideally < 15ms p50 |
| Personalization? | Yes — user history + global trends |
| Multi-language? | Yes — at least 10 major languages |
| Mobile? | Yes — both web and mobile clients |
| Update frequency? | Daily batch updates (hourly trending) |

**N INSIGHT**

Always clarify: query volume, latency requirements, update frequency, personalization scope, languages, and whether to support typo correction.


© Study Guide — For personal use only

# 02 Functional & Non-Functional Requirements

What the system must and should do

## Functional Requirements

- Given a partial query string, return top-k (typically 5–10) suggestions in real-time
- Support prefix matching (e.g., 'ap' → apple, app, application)
- Rank suggestions by popularity, relevance, and user history
- Update suggestions daily based on aggregate query trends
- Support multiple languages and alphabets (Latin, Cyrillic, CJK)

## Non-Functional Requirements

| Requirement | Target | Rationale |
|---|---|---|
| Latency (p99) | < 50 ms | Mobile users expect instant feedback |
| Availability | 99.95% | Global service, regional failover |
| QPS capacity | 58,000 QPS | 5B queries/day ÷ 86,400 sec |
| Query freshness | ≤ 1 min | Trending queries update hourly |
| Consistency | Eventual | Ok to have stale suggestions briefly |
| Cost efficiency | < 1 µ$ per query | Scale requires optimization |


© Study Guide — For personal use only

# 03 Capacity Estimation

The numbers that drive architecture decisions

## Query Volume Breakdown

```
Daily Queries: 5,000,000,000 (5B)
Seconds per day: 86,400
Queries per second: 57,870 QPS (≈ 58K QPS)

Hourly average: ~2.4M queries
Peak hour (4× avg): ~9.6M queries (≈ 111K QPS peak)
```

Peak QPS determines server count and cache sizing

## Cache Hit Ratio & Data Volume

1. **Cache hit ratio:** ~90% of requests hit Redis (popular prefixes)
2. **Top-k results per prefix:** 10 suggestions per prefix
3. **Unique prefixes:** Estimate 1–2M for English, scale to ~5–10M total
4. **Bytes per suggestion:** ~50–100 bytes (text + metadata)
5. **Cache size (Redis):** 5M prefixes × 10 results × 100 bytes = 5GB
6. **Trie storage (Cassandra):** ~750GB (full index + redundancy)

## Network & Storage

```
Avg query size: ~20 bytes (UTF-8 prefix)
Avg response size: ~500 bytes (10 results + metadata)
Network throughput (58K QPS):
Inbound: 58K × 20B = 1.16 MB/s
Outbound: 58K × 500B = 29 MB/s

Storage (Cassandra + replicas × 3):
Raw trie index: ~750 GB
With replication: ~2.25 TB across cluster
```

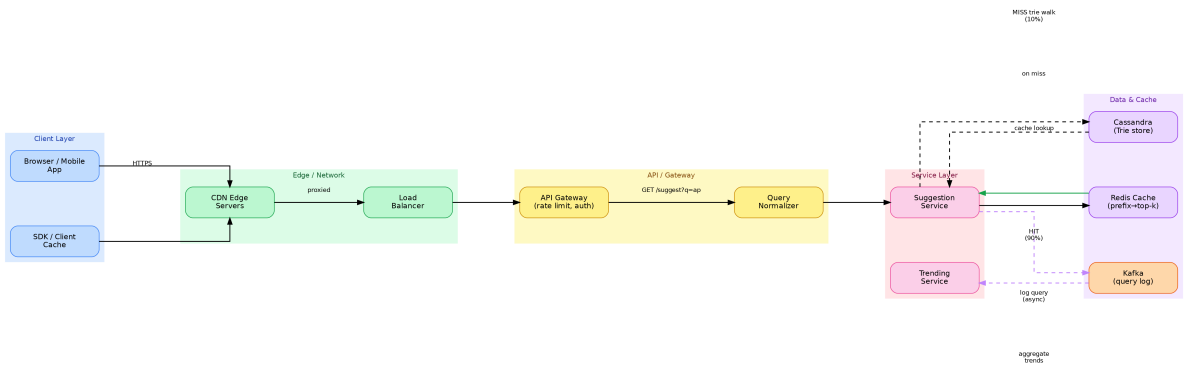
Network and storage are not bottlenecks; latency is king


© Study Guide — For personal use only

# 04 High-Level System Architecture

Five-tier system design with distributed caching


© Study Guide — For personal use only




**Architecture Diagram:** Five distinct tiers orchestrate the request flow. Client requests route through CDN → Load Balancer → API Gateway → Query Normalizer → Suggestion Service. The suggestion service queries Redis (90% cache hits) or falls through to Cassandra on miss. Async path logs queries to Kafka for trend analysis.


© Study Guide — For personal use only

## 05 Architecture Component Breakdown

Role and responsibility of each tier

### Client Layer

- **Browser / Mobile App:** UI initiates requests; implements client-side debouncing (50–100ms)
- **SDK / Client Cache:** Optional local cache for user's recent queries (very fast, <1ms)

### Edge / Network Layer

- **CDN Edge Servers:** Geographic distribution; serve cached responses for popular prefixes
- **Load Balancer:** Route traffic across API gateway instances; health-check based failover

### API & Gateway Layer

- **API Gateway:** Rate limiting (token bucket), auth validation, request logging
- **Query Normalizer:** Lowercase, trim whitespace, handle Unicode normalization

### Service Layer

- **Suggestion Service:** Core logic: query Redis cache → on miss, trie walk in Cassandra → rank results
- **Trending Service:** Async consumer of Kafka events; updates trending cache every hour

### Data & Cache Layer

- **Redis Cache:** In-memory store of top-10 suggestions per prefix; 10-minute TTL
- **Cassandra Cluster:** Immutable trie index; sharded by prefix hash; 3× replication
- **Kafka Topic:** Event log of all queries; feeds trending service and batch pipeline

> **N INSIGHT**
>
> Separation of concerns: read-path (cached) from write-path (offline batch). Eventual consistency model enables high availability.


© Study Guide — For personal use only

# 06 Core Data Structure: Trie

Prefix trees for efficient O(m) lookup

## Why a Trie?

- **Prefix matching:** O(m) to find all strings with prefix, where m is query length
- **Memory efficient:** Common prefixes share subtrees (e.g., 'app', 'apple' share 'app' node)
- **No full-text scan:** Direct navigation from root to prefix node
- **Ranked results:** Store top-k at each node (pre-computed during batch rebuild)
- **Immutable:** Each version is a snapshot; enables atomic blue-green swaps

## TrieNode Structure

```
class TrieNode:
def __init__(self):
self.children = {} # char → TrieNode
self.is_word = False
self.frequency = 0
self.top_k = [] # [(query, score, freq), ...]
self.personalized_top_k = {} # user_id → [(query, score), ...]
```

Each node caches top-k suggestions; updated during offline batch rebuild

## Lookup Complexity

| Operation | Complexity | Notes |
|---|---|---|
| Find prefix node | O(m) | m = length of query prefix |
| Get top-k results | O(1) | Cached at node |
| Insert/update | O(m) | Used only during batch rebuild |
| Serialization | O(n) | n = total trie nodes |


© Study Guide — For personal use only

# 07 Ranking & Scoring Formula

Popularity, recency, personalization, and context

## Multi-Signal Scoring

```
score = w1 × freq + w2 × recency + w3 × personalization + w4 × context

where:
freq = normalized query frequency (0–1)
recency = time decay (recent queries weighted higher)
personalization = user's historical preference (0–1)
context = language, region, device type, user location

Recommended weights: w1=0.4, w2=0.3, w3=0.2, w4=0.1
```

Weighted combination of multiple ranking signals

## Time Decay Function (Recency)

```
def recency_score(days_old):
# Half-life = 30 days; older queries decay exponentially
return 2 ** (-days_old / 30.0)

# Example:
# Today: score = 1.0
# 7 days ago: score ≈ 0.84
# 30 days ago: score = 0.5
```

Exponential decay ensures fresh trends rank higher

## Ranking Signals Comparison

| Signal | Latency | Accuracy | Update Freq | Personalization? |
|---|---|---|---|---|
| Query frequency | O(1) | High | Daily | Global |
| User history | O(1) | High | Real-time | Per-user |
| Contextual (region) | O(1) | Medium | Hourly | Per-region |
| ML ranker | ~5ms | Very high | Weekly | Per-user |
| Click-through rate | O(1) | Medium | Daily | Per-user |

**✓ TIP**

Start simple (frequency + recency), then layer personalization and ML ranking. Avoid high-latency signals in the hot path.

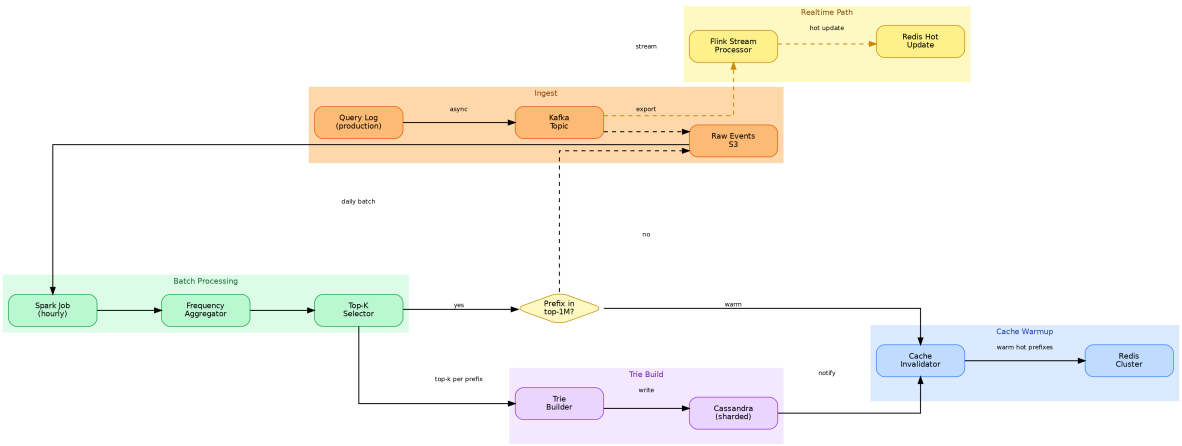

© Study Guide — For personal use only

© Study Guide — For personal use only

# 08 Trie Update Pipeline

Offline batch and realtime streaming paths


© Study Guide — For personal use only



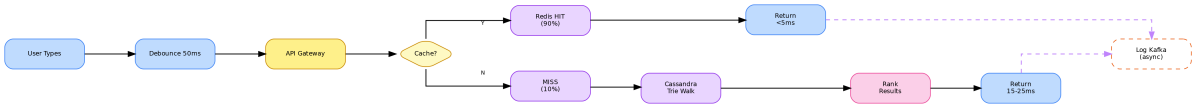

**Pipeline Diagram:** Queries flow to Kafka. Batch job (hourly) aggregates frequency, selects top-k per prefix, builds new trie, and writes to Cassandra staging. Decision diamond checks if prefix is in top-1M hot set; if yes, pre-warm Redis. Realtime path (Flink) streams hot updates to Redis for trending queries. Blue-green swap atomically promotes staging trie to live.


© Study Guide — For personal use only

# 09 Request Flow Deep Dive

**Cache hit vs. miss: latency breakdown and code flow**


© Study Guide — For personal use only




**Request Lifecycle:** User types → debounce (50ms) → API Gateway validation → cache lookup (Redis). Hit path returns top-10 in <5ms. Miss path: Cassandra trie walk (15–25ms), rank by frequency × recency × personalization, populate cache, return results. Async: log query to Kafka for analytics.


© Study Guide — For personal use only

# 10 Database Schema & Persistence

Cassandra for immutable trie snapshots

## Why Cassandra?

- **Column-family store:** Efficient for key-value lookups by prefix range
- **High availability:** Tunable consistency; QUORUM reads with 3× replication
- **Horizontal scaling:** Partition by prefix (consistent hashing); add nodes without reshuffling
- **Immutable snapshots:** Each trie version identified by timestamp; enables blue-green swap

## Cassandra Schema

```
CREATE TABLE typeahead.trie_index (
prefix TEXT, -- e.g., 'a', 'ap', 'app'
version BIGINT, -- trie version (timestamp)
top_k_results FROZEN<list>, -- [(query, freq, score), ...]
is_live BOOLEAN, -- current live version
PRIMARY KEY (prefix, version)
)
WITH CLUSTERING ORDER BY (version DESC)
AND compaction = {'class': 'LeveledCompactionStrategy'}
AND compression = {'sstable_compression': 'LZ4Compressor'};
```

Partition by prefix hash; version enables blue-green swaps and rollback

## Snapshot & Swap Strategy

1. **Serialization:** Protocol Buffers (compact, versioned, backwards-compatible)
2. **Rebuild schedule:** Daily batch (off-peak: 2 AM UTC) + hourly trending update
3. **Retention:** Keep 2 versions on disk: live and staged (backup just before swap)
4. **Atomic swap:** Update is_live flag; instant propagation via streaming replication
5. **Rollback:** Revert is_live flag; previous version still on disk


© Study Guide — For personal use only

# 11 Partitioning Strategy

**Consistent hashing for balanced distribution**

### Option 1: First-Letter Sharding (■ Poor)

- **Approach:** Shard by first letter of query (26 shards for English)
- **Problem:** Highly uneven distribution; 'a' prefix gets 10× more traffic than 'z'
- **Result:** Hot shards → high latency, difficult to rebalance dynamically

### Option 2: Consistent Hashing (✓ Recommended)

- **Approach:** Hash prefix using consistent hash ring; replicate on N successor nodes
- **Advantage:** Even distribution; incremental scaling (add nodes without reshuffling)
- **Implementation:** Jump consistent hash (Google) or Maglev hashing
- **Replication factor:** N=3 (write to primary + 2 successors; QUORUM=2)

### Consistent Hash Example

```
def consistent_hash(key: str, num_nodes: int) -> int:
"""Google's jump consistent hash for fast, even distribution."""
hash_value = hash(key) & 0x7fffffff # unsigned 31-bit
return hash_value % num_nodes

def get_replicas(prefix: str, num_nodes: int, replication: int = 3):
"""Get replica nodes for prefix."""
primary = consistent_hash(prefix, num_nodes)
return [primary] + [(primary + i) % num_nodes for i in range(1, replication)]
```

Consistent hashing avoids hot spots; scales horizontally


© Study Guide — For personal use only

# 12 Failure Modes & Recovery

Detecting and mitigating production incidents

## Failure Mode Analysis

| Failure | Detection | Mitigation | Recovery Time |
|---|---|---|---|
| Redis cluster down | Health check | Fall through to Cassandra; p99 ~40ms | ~1 min |
| Cassandra node down | QUORUM read fails | Serve from replicas (N-1) | ~2 min |
| Trie rebuild fails | CloudWatch alert | Keep previous version live | ~5 min |
| API Gateway overload | Request latency spike > 100ms | Rate limit + queue burst | Immediate |
| Network partition | Timeout > 2s | Fallback to cached results | ~10 sec |
| Slow disk I/O | Query latency p99 > 100ms | Increase cache TTL; shed load | ~5 min |
| Corrupted Cassandra data | Checksum mismatch | Restore from backup | ~30 min |
| DDoS attack on API | QPS spike > 2× normal | IP-based rate limiting | ~2 min |

## Monitoring & Alerting

- **Key metrics:** QPS, p50/p99 latency, cache hit ratio, error rate, trie staleness
- **Dashboards:** Real-time SLO tracking (Grafana + Prometheus)
- **Alerting:** Auto-page on-call if p99 > 50ms or error rate > 0.1%
- **Postmortems:** Blameless root-cause analysis on every incident > 10 min downtime


© Study Guide — For personal use only

# 13 Interview Playbook (45 min)

Walkthrough timeline and talking points

## 45-Minute Timeline

| Time | Section | Key Points |
|---|---|---|
| 0–5 min | Clarification | Ask about scale (5B QPS?), latency SLA (<50ms?), personalization, languages |
| 5–10 min | Requirements | Func/non-func; list main components: cache, trie, partitioning |
| 10–20 min | Architecture | Whiteboard high-level diagram; explain: client → cache → trie → DB |
| 20–30 min | Deep dive | Trie structure (O(m) lookup), caching (90% hits), ranking (freq × recency) |
| 30–40 min | Scale-up | Partitioning (consistent hashing), failure modes, trade-offs |
| 40–45 min | Polish | Recap design, ask for feedback, mention future improvements |

## Must-Know Numbers

```
Scale: 5 billion queries/day → ~58,000 QPS (peak 111K QPS)
Trie size: ~750 GB raw (2.25 TB with 3× replication)
Cache size: ~5 GB (5M prefixes × 10 results × 100 bytes)
Cache hit ratio: ~90% (90% from Redis, 10% from Cassandra)
p99 latency: ~15 ms (5ms Redis + network + API overhead)
Cache TTL: 10 minutes (freshness vs. hit rate trade-off)
Unique prefixes: ~5–10 million (across all languages)
```

Memorize these numbers; they validate your design decisions

## Impressive Talking Points

- **Eventual consistency:** 'Trie updates are atomic snapshots; we swap versions instantly. Clients tolerate brief staleness (<1 day) for high availability.'
- **Cache warming:** 'Pre-load 1M hot prefixes at startup. Use lazy-loading for cold prefixes. Decision diamond in pipeline: is prefix in top-1M? If yes, warm cache.'
- **Consistent hashing:** 'Prefixes distributed evenly across nodes via jump hash. Add nodes incrementally without reshuffling; 3× replication for fault tolerance.'
- **Failure resilience:** 'Redis miss → Cassandra fallback (~40ms p99). Cassandra down → serve stale results from local pod cache. Trie rebuild fails → keep previous version live.'
- **Scalability:** 'Batch pipeline (hourly) aggregates 5B queries into top-k per prefix. Realtime path (Flink) streams hot updates. Both paths converge on Redis → Cassandra.'


© Study Guide — For personal use only

### Expected Follow-Up Questions

- **How do you handle typos?** 'Edit-distance algorithm + separate typo-correction Trie. Query both tries and merge results. Trade-off: ~5ms additional latency.'
- **Multi-word queries?** 'Tokenize query; walk trie per token; merge results by combined frequency.'
- **100× traffic surge?** 'Rate limiting on API Gateway (token bucket). Shed load: return partial results. Auto-scale pods by QPS (k8s HPA). Cache becomes warmer; hit ratio improves.'
- **Cost optimization?** 'Compress Cassandra (LZ4). Use spot instances for batch jobs. Leverage CDN for edge caching. Aggressive cache TTLs on hot prefixes.'


© Study Guide — For personal use only

# 14 Key Takeaways

Core insights and design principles

**N INSIGHT**

**1. Prefix Trie for O(m) Lookup:** Pre-compute top-k at each node. Immutable snapshots enable atomic version swaps.

**2. Multi-Tier Caching:** Client → CDN → Redis (90% hits) → Cassandra (10% misses). Cache hit in <5ms; miss in 15–25ms. Eventual consistency acceptable.

**3. Separation of Read & Write Paths:** Read: synchronous, cached, latency-sensitive. Write: offline batch (daily) + realtime streaming (hourly). Never block reads on writes.

**4. Consistent Hashing for Scale:** Distribute prefixes evenly. Add nodes incrementally. 3× replication provides fault tolerance and data locality.

**5. Rank by Signals:** Start with frequency + recency (fast). Layer personalization (cached, 1-hour TTL). Avoid expensive ML rankers in critical path; use for offline ranking.

**6. Monitoring & Resilience:** Track latency (p50/p99), cache hit ratio, error rate. Design for graceful degradation: Redis down → use Cassandra; Cassandra down → use stale cache.


© Study Guide — For personal use only